7-2 · 텍스트 데이터 Chunking · 실습 1.5H

Document Loader와 텍스트 분할 실습

문서를 읽고 자르고, 잘린 경계를 눈으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① 네 장짜리 notes.txt(642자)를 만들어 TextLoader 로 읽는다

② 일부러 cp949 로 읽어 감싼 오류 속 진짜 원인을 찾는다

③ 200 · 30 으로 잘라 조각 다섯 개의 경계와 겹침을 본다

④ 메타데이터를 확인하고 Character 분할기와 비교한다

파일마다 cat > 파일.py << 'EOF' … EOF 로 만들고 python 파일.py 로 실행합니다

실습 흐름

일곱 단계 — ~/data-eng/week7 한 폴더에서

7-1 의 가상환경을 그대로 켜고 시작합니다 — Windows(WSL2)·macOS·리눅스 모두 같습니다

1. 문서	2·3. 로드·실패	4. 분할	5·6. 겹침·메타	7. 비교
notes.txt 642자 · 네 장	load.py enc.py(cp949)	split.py 200 / 30	overlap.py meta.py	compare.py Character

1단계

실습 문서 notes.txt 를 만드세요

1. **cd ~/data-eng/week7 → source .venv/bin/activate**
   7-1 의 가상환경 — 패키지는 이미 설치돼 있다

2. **cat > notes.txt << 'EOF' ... EOF**
   파이프라인 개요 · 오케스트레이션 · 스트리밍 · 저장 네 장

3. **wc -m notes.txt**
   642 notes.txt — 글자 수, 한글도 한 글자(실측)

4. **2장 Airflow 문단은 일부러 228자**
   200자보다 길다 — 4 · 7단계에서 쓰인다

글자 수

642자

2단계

load.py — 파일 하나를 Document 하나로

```
cat > load.py << 'EOF'
from langchain_community.document_loaders import TextLoader

docs = TextLoader("notes.txt", encoding="utf-8").load()
print(len(docs))                # 파일 1개 = Document 1개
print(docs[0].metadata)         # {'source': 'notes.txt'}
print(docs[0].page_content[:80])
EOF
python load.py
```

실측 1 · {'source': 'notes.txt'} · 「1장 데이터 파이프라인 개요」— 맨 위 DeprecationWarning 두 줄은 무시

3단계 — 일부러 실패시키기

cp949 로 읽으면 — 감싼 오류 속 진짜 원인

enc.py 가 e 와 e.__cause__ 를 함께 찍습니다 — 오류 이름만 보지 말고 원인 줄까지 읽으세요

1. 틀리게 읽기	2. 겉 오류	3. 진짜 원인	4. 교훈
TextLoader encoding="cp949"	RuntimeError Error loading	e.__cause__ UnicodeDecodeError	원인 줄까지 읽기 encoding 명시

4단계

split.py — 200자 조각 다섯 개

chunk	길이	들어 있는 것
0	189자	1장 + 「2장 오케스트레이션」 제목만
1	198자	Airflow 문단 앞부분
2	58자	Airflow 문단 나머지
3	145자	3장 스트리밍 + 「4장 저장」
4	80자	4장 저장

① 제목이 본문과 떨어진 곳이 있다 ② 228자 Airflow 문단은 공백 경계로 둘(198 · 58)로 나뉘었다

조각 수: 5 · 모두 200자 이하(실측) — 대부분 문단 끝에서 잘렸다

5단계

overlap.py — 겹친 곳은 한 군데

[1] 끝 ↔ [2] 시작 — 겹침

문단 안에서 자른 곳 — overlap 30자가 다시 붙었다

- …걸린 시간과 성공 여부가 기록으로 남아, 어제 어느
- 걸린 시간과 성공 여부가 기록으로 남아, 어제 어느 …
- 같은 글이 양쪽에 들어 있다(실측)

VS

나머지 경계 — 겹침 없음

[0]→[1] · [2]→[3] · [3]→[4] 는 문단 경계에서 딱 나뉨

- 이어 줄 문장이 필요 없었다
- 다만 「4장 저장」은 [3] 끝 · [4] 시작 양쪽에
- 끝 40자 · 시작 40자를 나란히 찍어 대조

6단계

meta.py — 메타데이터가 따라오는지, 직접 더하기

1. **print(i, c.metadata) — 다섯 줄**
   모두 {'source': 'notes.txt'} — 자른 뒤에도 출처가 조각마다 따라온다

2. **c.metadata["chunk_id"] = i · ["chars"] = len(...)**
   마지막 줄 {'source': 'notes.txt', 'chunk_id': 0, 'chars': 189}(실측)

3. **7-3 에서 쓰임**
   벡터DB 에 넣을 때 payload 로 함께 들어간다 — PDF 라면 page 도

출처의 쓸모

어느 문서

몇 쪽

7단계

compare.py — Recursive 가 기본인 이유

Recursive — [189, 198, 58, 145, 80]

줄 · 공백으로 내려가 끝까지 200자를 지킨다



VS

Character — [189, 228, 145, 80]

구분자 \n\n 하나뿐 — 228자 문단을 못 자른다

- separator="\n\n" · 200 / 30 은 같게
- Created a chunk of size 228, which is longer than the specified 200
- 200자를 넘는 조각이 그대로 나온다

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. wc -m 이 642 notes.txt 를 보인다
2. load.py 가 1 · {'source': 'notes.txt'} 를 찍는다
3. cp949 로 읽으면 RuntimeError, 원인은 UnicodeDecodeError
4. 조각 수 5 — 189 · 198 · 58 · 145 · 80 자
5. [1] 끝과 [2] 시작에 같은 글이 겹쳐 있다
6. Character 분할기는 228자 조각을 그대로 낸다

7-2 · 실습 1.5H 마무리

다음은 Chunk 파라미터 튜닝 (1H)

여덟 절짜리 긴 문서로 size·overlap 조합을 재 보고, 우리 문서에 맞는 기준값을 근거와 함께 정합니다

- ✓ long_notes.txt 여덟 절 · 1,247자
- ✓ 다섯 조합의 조각 수 · 평균 길이
- ✓ 문장 중간 절단 · 「CDC」 조각 비교
- ✓ Markdown 제목 우선 · 기준값 기록

남겨 둘 것

notes.txt — 7-3 부터 검색 대상